

核心数据结构图解

struct iio_dev 与 pollfunc

IIO 设备的"总枢纽"与触发缓冲的"工单" —— 内部结构 · 字段 · 创建

MPU-6050 IIO 驱动学习 · 数据结构篇 · 2026 年 10 月

导读

触发缓冲驱动里有两个结构体最该吃透: `struct iio_dev` (整个 IIO 设备的总枢纽,probe 里填它、注册它) 和 `struct iio_poll_func (pollfunc)`(触发器与缓冲之间的"工单")。本文用图+表把它们的**内部结构、每个字段、在哪创建**讲清。

一、struct iio_dev:IIO 设备的总枢纽

`indio_dev` 是指向 `struct iio_dev` 的指针。这个结构体把"设备是谁、能干啥、怎么读、运行时挂了哪些机制"全登记在一起。按作用分四组看:

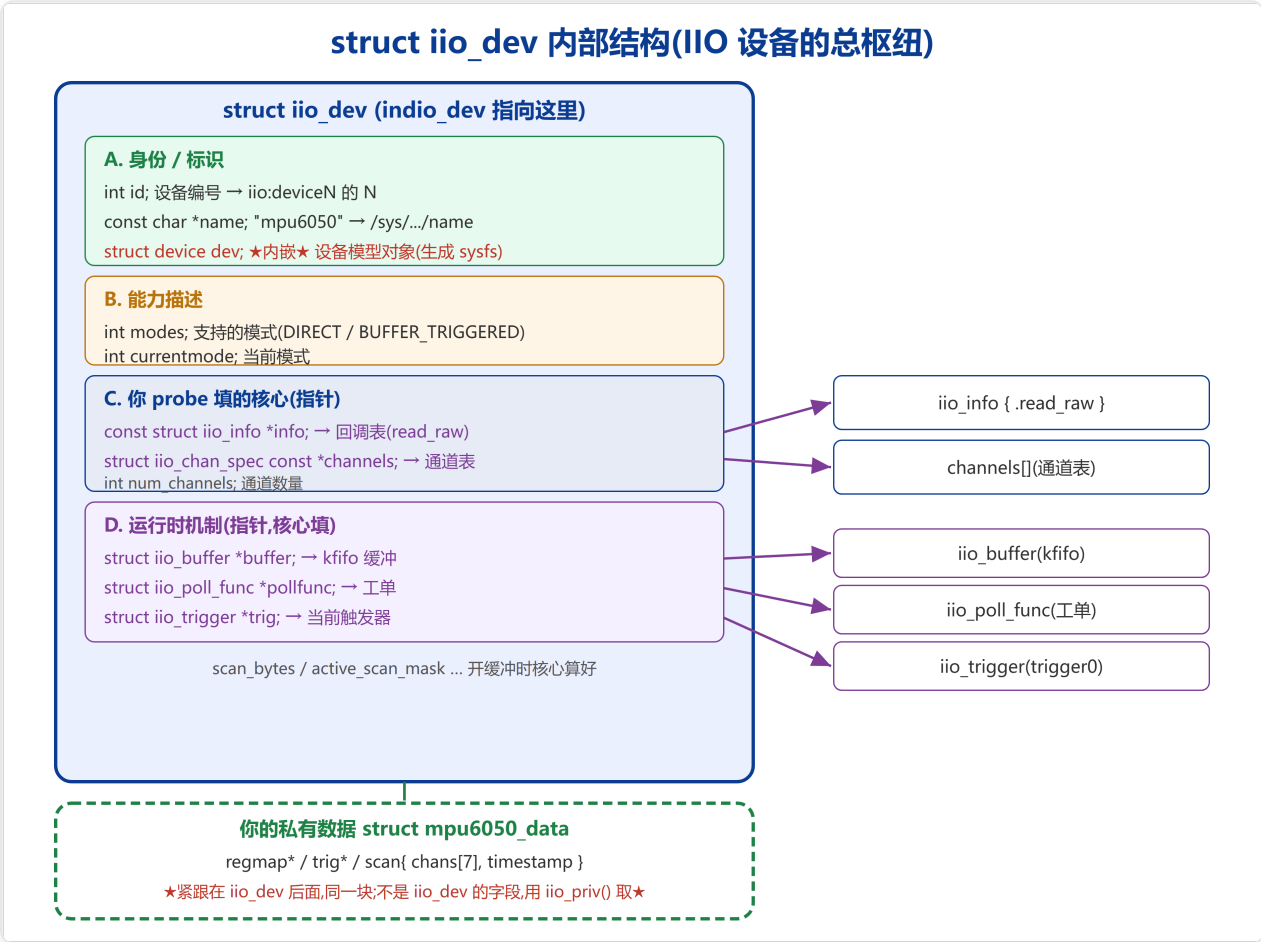


图 1 struct iio_dev 内部结构(四组字段 + 指向的外部对象 + 尾随的私有数据)

组	字段	作用
A 身份	id	第几个 IIO 设备 → iio:device0 的 0;触发器名 "%s-dev%d" 的 %d

	name	设备名 "mpu6050" → /sys/.../name
	dev	内嵌 的 struct device (Linux 设备模型对象,sysfs 目录由它生成)
B 能力	modes	支持的模式: INDIO_DIRECT_MODE ;触发缓冲补 INDIO_BUFFER_TRIGGERED
	currentmode	当前所处模式(核心维护)
C 核心 (你填)	info	指向 iio_info 回调表(含 read_raw)
	channels / num_channels	指向通道表+数量 → 核心据此生成 sysfs、解析缓冲帧
D 运行时	buffer / pollfunc / trig	指向 kfifo 缓冲 / 工单 / 当前触发器(核心填)
	scan_bytes / active_scan_mask	开缓冲时核心算:选了哪些通道、一帧多少字节

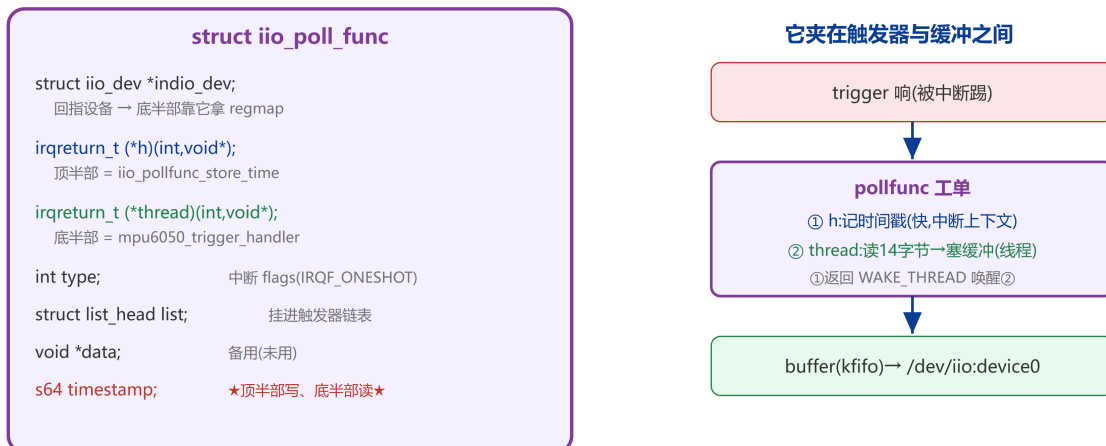
三种"住处"要分清:① **内嵌**(值/对象就在结构体里):id、name、modes、dev;② **指针**(指向外面独立的块):info、channels、buffer、pollfunc、trig;③ **私有数据** mpu6050_data **不是 iio_dev 的字段**,而是 devm_iio_device_alloc 时紧跟其后的一段,用 iio_priv(indio_dev) 取。

所以 probe 的本质就是:**填好 C 组(info/channels)→ 建起 D 组(缓冲/触发器)→ 注册**。核心运行时就靠读这张"登记表"来回调你、生成接口、搬数据。

二、pollfunc:触发器与缓冲之间的"工单"

触发器只会喊"滴答",它不知道该读什么;缓冲是目标,但得有人去填。`pollfunc` 就是夹在中间的工单,写明"滴答一到,先记时间,再读一帧塞进缓冲"。

pollfunc(工单):结构 + 它在哪一环



运行时接力(为什么分两半)

- ① h 在中断上下文,必须极快、不能睡眠 → 只记 timestamp,返回 IRQ_WAKE_THREAD
- ② thread 在内核线程,可睡眠 → regmap_bulk_read 读 I2C + iio_push_to_buffers_with_timestamp(用 pf->timestamp)

一句话:pollfunc 把"记时间"和"读数据塞缓冲"两个函数打包成一张工单

触发器一响就按工单执行;时间戳格子让两半接力(①写②读)
存在 indio_dev->pollfunc,是独立一块内存

图 2 pollfunc 的结构(左)与它在"触发器→缓冲"链条中的位置(右)

2.1 结构字段

```
struct iio_poll_func {
    struct iio_dev *indio_dev;           // 回指设备 → 底半部靠它拿 regmap
    irqreturn_t (*h)(int irq, void *p); // 顶半部 = iio_pollfunc_store_time
    irqreturn_t (*thread)(int irq, void *p); // 底半部 = mpu6050_trigger_handler
    int type;                             // 中断 flags(IRQF_ONESHOT)
    struct list_head list;                 // 挂进触发器的消费者链表
    void *data;                             // 备用(未用)
    s64 timestamp;                         // ★顶半部写、底半部读★
};
```

字段	在本项目是谁	作用
indio_dev	iio:device0	底半部 iio_priv 拿到 regmap

h	iio_pollfunc_store_time	滴答瞬间记时间戳到 timestamp (快,中断上下文)
thread	mpu6050_trigger_handler	线程里 regmap_bulk_read + push 进缓冲(可睡眠)
type	IRQF_ONESHOT	决定这张工单的虚拟中断怎么申请
timestamp	—	顶半部写、底半部读: iio_push_...(..., pf->timestamp)

2.2 两半如何接力

绑定到触发器时,核心按"线程化中断"把 h 当硬处理、thread 当线程函数。触发器一响:① h 记时间戳,返回 IRQ_WAKE_THREAD 唤醒线程 → ② thread 在线程里读 I2C、push 进缓冲(用 ① 记下的 pf->timestamp)。拆两半是因为:① 在中断上下文必须极快、不能睡眠;② I2C 读会睡眠,必须放线程——这正是内核"中断上下半部"的标准分工。

2.3 它在哪被创建

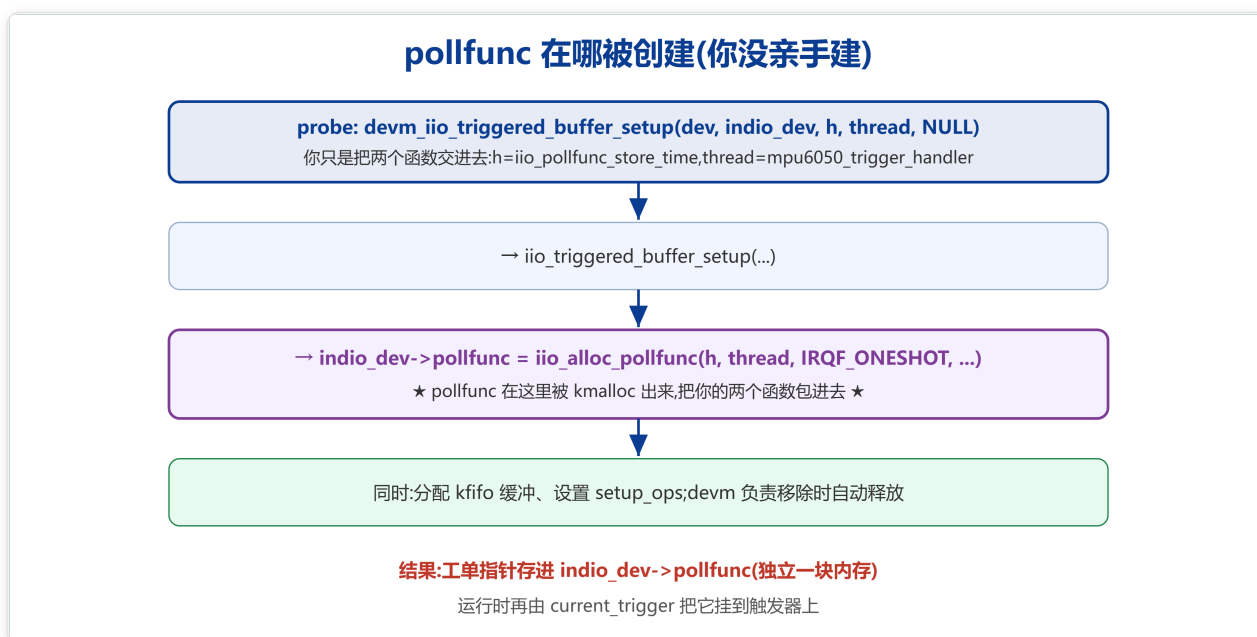


图3 pollfunc 的创建调用链:你传两个函数进去,核心帮你建

你**不直接** new 一个 pollfunc,而是把两个函数交给 devm_iio_triggered_buffer_setup,它内部 iio_alloc_pollfunc(h, thread, IRQF_ONESHOT, ...) 把它们包进一张新工单,指针存进 indio_dev->pollfunc (独立一块内存);同时还分配 kfifo 缓冲。devm_ 负责移除时自动释放。运行时再由 current_trigger 把这张工单挂到触发器上。

三、两者的关系(收口)

indio_dev 是总枢纽,pollfunc 是它 D 组里的一个成员(`indio_dev->pollfunc`)。probe 填好 iio_dev 的 C 组、建起 D 组(其中 `devm_iio_triggered_buffer_setup` 顺带造出 pollfunc 和缓冲)、再注册。运行时核心通过 iio_dev 这张登记表找到 info/channels/buffer/trig/pollfunc,把整条"触发→工单→缓冲"的数据流跑起来。

本文为 MPU-6050 IIO 驱动学习的数据结构篇,承接剖析篇。系列文档见 [docs/learning/](#) ;配套代码见 [examples/learning-driver/](#) 。仓库:github.com/fyy0619-cell/xiangmu